

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Tiểu luận khoa học

ỨNG DỤNG
CÁC MÔ HÌNH QUY HOẠCH TUYẾN TÍNH
CHO CÁC LOẠI BÀI TOÁN KNAPSACK

Nguyễn Hoàng Đạt - 24001989
Nguyễn Hải Anh - <Mã SV>
Lê Sỹ Thức - <Mã SV>

Ngành: Khoa học máy tính và thông tin

Hà Nội, 2026

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Tiểu luận khoa học

ỨNG DỤNG
CÁC MÔ HÌNH QUY HOẠCH TUYẾN TÍNH
CHO CÁC LOẠI BÀI TOÁN KNAPSACK

Nguyễn Hoàng Đạt - 24001989
Nguyễn Hải Anh - <Mã SV>
Lê Sỹ Thức - <Mã SV>

Ngành: Khoa học máy tính và thông tin

Hà Nội, 2026

Tóm tắt nội dung

Ra đời vào năm 1947 và được xem là một trong những thuật toán có ảnh hưởng sâu rộng nhất của thế kỷ XX, thuật toán đơn hình (simplex method) đóng vai trò nền tảng trong lĩnh vực quy hoạch tuyến tính và tối ưu hóa. Sự phát triển của các biến thể như thuật toán đơn hình hai pha và thuật toán đơn hình đối ngẫu đã mở rộng đáng kể khả năng ứng dụng của phương pháp này trong nhiều bài toán tối ưu thực tiễn thuộc khoa học, kỹ thuật và công nghiệp.

Bài toán cái túi (Knapsack Problem) là một trong những bài toán tối ưu tổ hợp kinh điển của khoa học máy tính, với nhiều ứng dụng trong phân bổ tài nguyên, quản lý danh mục đầu tư, lập lịch sản xuất và hỗ trợ ra quyết định. Trong báo cáo này, chúng tôi khảo sát ba biến thể quan trọng của bài toán knapsack gồm fractional knapsack, unbounded knapsack và 0/1 knapsack dưới góc nhìn của quy hoạch tuyến tính và quy hoạch nguyên. Trên cơ sở đó, báo cáo trình bày việc áp dụng các phương pháp đơn hình, bao gồm đơn hình nguyên thủy, đơn hình hai pha và đơn hình đối ngẫu, để giải các mô hình tương ứng.

Ngoài việc phân tích mô hình toán học và cơ chế hoạt động của các thuật toán, báo cáo còn đánh giá hiệu quả thực nghiệm của các phương pháp dựa trên quy hoạch tuyến tính, đồng thời so sánh chúng với các chiến lược thiết kế thuật toán kinh điển như vét cạn, quay lui, nhánh-cận, tham lam và quy hoạch động. Qua đó, báo cáo làm rõ ưu điểm, hạn chế cũng như tiềm năng ứng dụng của các phương pháp đơn hình trong việc giải các bài toán knapsack và các bài toán tối ưu tổ hợp mở rộng.

Mục lục

1	Giới thiệu	1
2	Một số loại bài toán Knapsack	1
2.1	0/1 Knapsack	1
2.2	Unbounded Knapsack	2
2.3	Fractional Knapsack	2
3	Các chiến lược cơ bản	3
3.1	Vét cạn và Quay lui	3
3.2	Nhánh cận (Branch and Bound)	3
3.3	Tham lam (Greedy)	4
3.4	Quy hoạch động (Dynamic Programming)	4
4	Quy hoạch tuyến tính trong bài toán fractional knapsack	5
4.1	Chuẩn hóa bài toán và dạng chuẩn	5
4.2	Thuật toán đơn hình hai pha	5
4.3	Thuật toán đơn hình đối ngẫu	6
4.4	Cập nhật nghiệm tối ưu khi bài toán thay đổi	6
4.5	Klee–Minty và tính mũ của đơn hình	6
5	Quy hoạch nguyên trong bài toán unbounded knapsack và 0/1 knapsack	7
5.1	Ý tưởng và mô hình toán học	7
5.2	Phương pháp nhánh-cận (Branch and Bound)	7
5.3	Lát cắt Gomory	7
6	Thực nghiệm, so sánh và đánh giá	7
6.1	Phương pháp sinh dữ liệu và thiết lập thực nghiệm	8
6.2	Kết quả thực nghiệm	10
6.3	Thảo luận	14
7	Kết luận	15

1 Giới thiệu

Bài toán cái túi (Knapsack Problem) là một trong những bài toán tối ưu tổ hợp nền tảng trong khoa học máy tính và vận trù học. Bài toán được phát biểu đơn giản: cho một tập các vật phẩm, mỗi vật phẩm có trọng lượng và giá trị riêng, hãy chọn một tập con các vật phẩm sao cho tổng giá trị là lớn nhất mà tổng trọng lượng không vượt quá sức chứa của cái túi.

Mặc dù phát biểu đơn giản, bài toán Knapsack thuộc lớp NP-hard [?], nghĩa là chưa tồn tại thuật toán nào giải được mọi trường hợp trong thời gian đa thức (trừ khi $P = NP$). Điều này khiến bài toán trở thành đối tượng nghiên cứu quan trọng về mặt lý thuyết lẫn thực tiễn.

Ứng dụng thực tiễn. Bài toán Knapsack xuất hiện trong nhiều lĩnh vực:

- *Phân bổ tài nguyên:* phân bổ ngân sách, lập lịch dự án với tài nguyên giới hạn.
- *Quản lý danh mục đầu tư:* chọn tổ hợp tài sản tối ưu với vốn giới hạn.
- *Logistics:* xếp hàng vào container, xe tải với trọng tải giới hạn.
- *Mật mã học:* hệ mã Merkle–Hellman dựa trên bài toán Knapsack.

Mục tiêu đề tài. Báo cáo này tiếp cận bài toán Knapsack từ góc độ *quy hoạch tuyến tính* (LP) và *quy hoạch nguyên* (IP). Cụ thể, chúng tôi:

1. Trình bày ba biến thể chính: 0/1, Unbounded và Fractional Knapsack.
2. Phân tích các chiến lược giải cơ bản: vét cạn, quay lui, nhánh cận, tham lam, quy hoạch động.
3. Xây dựng mô hình LP/IP và áp dụng các thuật toán Simplex, lời giải đối ngẫu, lát cắt Gomory.
4. Thực nghiệm so sánh hiệu năng trên bộ dữ liệu benchmark với các mức độ phức tạp khác nhau.

2 Một số loại bài toán Knapsack

Trong phần này, chúng tôi trình bày ba biến thể phổ biến nhất của bài toán Knapsack [?]. Sự khác biệt cốt lõi giữa chúng nằm ở **miền giá trị của biến quyết định** x_i : nhị phân (0/1), nguyên không âm (Unbounded), hoặc liên tục trên $[0, 1]$ (Fractional).

Xét bài toán tổng quát với n vật phẩm, vật phẩm thứ i có trọng lượng $w_i > 0$, giá trị $v_i > 0$, và cái túi có sức chứa $W > 0$.

2.1 0/1 Knapsack

Trong bài toán 0/1 Knapsack, mỗi vật phẩm chỉ có thể được **chọn hoặc không chọn** (không chia nhỏ, không lặp lại). Biến quyết định $x_i \in \{0, 1\}$.

Mô hình toán học:

$$\text{Maximize} \quad \sum_{i=1}^n v_i \cdot x_i \quad (1)$$

$$\text{subject to} \quad \sum_{i=1}^n w_i \cdot x_i \leq W \quad (2)$$

$$x_i \in \{0, 1\}, \quad \forall i = 1, \dots, n \quad (3)$$

Đây là một bài toán **quy hoạch nguyên nhị phân** (Binary Integer Programming). Ràng buộc (3) khiến bài toán thuộc lớp NP-hard [?], do không gian tìm kiếm có kích thước 2^n .

Ví dụ. Cho 4 vật phẩm và sức chứa $W = 5$:

Vật phẩm i	A	B	C	D
Trọng lượng w_i	2	1	3	2
Giá trị v_i	12	10	20	15
Mật độ v_i/w_i	6.0	10.0	6.67	7.5

Bảng 1: Dữ liệu ví dụ cho bài toán Knapsack

Nghiệm tối ưu: chọn $\{A, B, D\}$ với tổng trọng lượng $2 + 1 + 2 = 5 = W$, tổng giá trị $12 + 10 + 15 = 37$.

2.2 Unbounded Knapsack

Trong bài toán Unbounded Knapsack, mỗi vật phẩm có thể được chọn **không giới hạn số lần**. Biến quyết định $x_i \in \mathbb{Z}_{\geq 0}$ (số nguyên không âm).

Mô hình toán học:

$$\text{Maximize } \sum_{i=1}^n v_i \cdot x_i \quad (4)$$

$$\text{subject to } \sum_{i=1}^n w_i \cdot x_i \leq W \quad (5)$$

$$x_i \in \mathbb{Z}_{\geq 0}, \quad \forall i = 1, \dots, n \quad (6)$$

So với 0/1 Knapsack, miền biến mở rộng từ $\{0, 1\}$ sang $\mathbb{Z}_{\geq 0}$, cho phép khai thác triệt để các vật phẩm có mật độ giá trị cao. Bài toán vẫn thuộc lớp NP-hard, tuy nhiên có thể giải bằng quy hoạch động với độ phức tạp giả đa thức $O(nW)$.

Ví dụ. Với dữ liệu Bảng 1 và $W = 5$: vật phẩm B có mật độ cao nhất ($v/w = 10$), chọn B $\times 5$ lần cho trọng lượng $1 \times 5 = 5$ và giá trị $10 \times 5 = 50$.

2.3 Fractional Knapsack

Trong bài toán Fractional Knapsack, mỗi vật phẩm có thể được chọn **một phần** (phân số từ 0 đến 1). Biến quyết định $x_i \in [0, 1]$.

Mô hình toán học:

$$\text{Maximize } \sum_{i=1}^n v_i \cdot x_i \quad (7)$$

$$\text{subject to } \sum_{i=1}^n w_i \cdot x_i \leq W \quad (8)$$

$$0 \leq x_i \leq 1, \quad \forall i = 1, \dots, n \quad (9)$$

Đây chính là bài toán **quy hoạch tuyến tính** (LP). Khác biệt quan trọng: miền biến liên tục $[0, 1]$ thay vì rời rạc, do đó bài toán giải được trong **thời gian đa thức** bằng chiến lược tham lam hoặc thuật toán Simplex [?].

Fractional Knapsack đóng vai trò quan trọng như **LP relaxation** của bài toán 0/1 Knapsack: nới lỏng ràng buộc $x_i \in \{0, 1\}$ thành $x_i \in [0, 1]$. Nghiệm tối ưu của LP relaxation luôn $\geq$ nghiệm tối ưu 0/1, tạo cận trên cho thuật toán Nhánh cận.

Ví dụ. Với dữ liệu Bảng 1 và $W = 5$: sắp xếp theo mật độ giảm dần $B(10) > D(7.5) > C(6.67) > A(6)$, chọn lần lượt:

- B toàn bộ ($x_B = 1$): trọng lượng 1, còn lại 4.
- D toàn bộ ($x_D = 1$): trọng lượng 2, còn lại 2.
- C một phần ($x_C = 2/3$): trọng lượng $3 \times 2/3 = 2$, còn lại 0.

Tổng giá trị: $10 + 15 + 20 \times 2/3 = 38.33$. Lưu ý giá trị này cao hơn nghiệm 0/1 tối ưu (37).

So sánh tổng hợp. Bảng 2 tóm tắt sự khác biệt giữa ba biến thể.

Đặc điểm	0/1	Unbounded	Fractional
Miền biến x_i	$\{0, 1\}$	$\mathbb{Z}_{\geq 0}$	$[0, 1]$
Loại mô hình	ILP	IP	LP
Độ phức tạp	NP-hard	NP-hard	$O(n \log n)$
Nghiệm ví dụ	37	50	38.33

Bảng 2: So sánh ba biến thể bài toán Knapsack

3 Các chiến lược cơ bản

Phần này trình bày các chiến lược giải cơ bản cho bài toán Knapsack. Với mỗi chiến lược, chúng tôi phân tích: (1) ý tưởng, (2) mã giả, (3) độ phức tạp thời gian và bộ nhớ, (4) tính chính xác của nghiệm.

3.1 Vét cạn và Quay lui

Vét cạn (Brute Force)

Ý tưởng. Liệt kê tất cả 2^n tập con của n vật phẩm, kiểm tra tính khả thi (tổng trọng lượng $\leq W$), và chọn tập con có giá trị lớn nhất.

- **Độ phức tạp thời gian:** $O(2^n)$ – duyệt tất cả tập con.
- **Độ phức tạp bộ nhớ:** $O(n)$ – lưu tập con hiện tại.
- **Tính chính xác:** Luôn cho nghiệm tối ưu.

Quay lui (Backtracking)

Ý tưởng. Cải tiến vét cạn bằng cách xây dựng nghiệm dần dần trên cây nhị phân (mỗi nút ứng với quyết định chọn/không chọn vật phẩm i), kết hợp **cắt tỉa** (pruning) để loại bỏ sớm các nhánh không triển vọng.

Algorithm 3.1 Quay lui cho 0/1 Knapsack

Input: Vật phẩm $(w_i, v_i)_{i=1}^n$, sức chứa W

Output: Giá trị tối ưu V^* , tập chọn S^*

$V^* \leftarrow 0$; $S^* \leftarrow \emptyset$ Tính trước: $\text{suffix}[i] = \sum_{j=i}^n v_j$

Function Backtrack(i, w, v, S): **if** $v > V^*$ **then** $V^* \leftarrow v$; $S^* \leftarrow S$

if $i > n$ **then return**

if $v + \text{suffix}[i] \leq V^*$ **then return**

if $w + w_i \leq W$ **then** Backtrack($i + 1, w + w_i, v + v_i, S \cup \{i\}$)

Backtrack($i + 1, w, v, S$)

Backtrack(1, 0, 0, $\emptyset$) **return** V^*, S^*

- **Độ phức tạp thời gian:** $O(2^n)$ trường hợp xấu nhất.
- **Độ phức tạp bộ nhớ:** $O(n)$.
- **Tính chính xác:** Luôn cho nghiệm tối ưu.

3.2 Nhánh cận (Branch and Bound)

Ý tưởng. Nhánh cận [?] cải tiến quay lui bằng cách sử dụng **cận trên** (upper bound) chặt hơn để cắt nhánh dựa trên LP relaxation.

Algorithm 3.2 Nhánh cận cho 0/1 Knapsack

Input: Vật phẩm đã sắp xếp giảm dần theo v_i/w_i , sức chứa W

Output: Giá trị tối ưu V^* , tập chọn S^*

$V^* \leftarrow 0$; $S^* \leftarrow \emptyset$

Function UpperBound(i, w, v): $\text{bound} \leftarrow v$; $\text{rem} \leftarrow W - w$ **for** $j \leftarrow i$ **to** n **do** **if** $w_j \leq \text{rem}$ **then** $\text{rem} \leftarrow \text{rem} - w_j$; $\text{bound} \leftarrow \text{bound} + v_j$ **else** $\text{bound} \leftarrow \text{bound} + v_j \times \text{rem}/w_j$; **break**

return bound

Function Branch(i, w, v, S): **if** $v > V^*$ **then** $V^* \leftarrow v$; $S^* \leftarrow S$

if $i > n$ **then return**

if $w + w_i \leq W$ **then** Branch($i + 1, w + w_i, v + v_i, S \cup \{i\}$)

if UpperBound($i + 1, w, v$) $> V^*$ **then** Branch($i + 1, w, v, S$)

Branch(1, 0, 0, $\emptyset$) **return** V^*, S^*

- **Độ phức tạp thời gian:** $O(2^n)$ trường hợp xấu nhất, thực tế nhanh hơn nhiều.

- **Độ phức tạp bộ nhớ:** $O(n)$.
- **Tính chính xác:** Luôn cho nghiệm tối ưu.

3.3 Tham lam (Greedy)

Ý tưởng. Sắp xếp vật phẩm theo **mật độ giá trị** v_i/w_i giảm dần, sau đó chọn lần lượt cho đến khi hết sức chứa. Tối ưu cho Fractional, heuristic cho 0/1.

Algorithm 3.3 Tham lam cho Fractional Knapsack

Input: Vật phẩm $(w_i, v_i)_{i=1}^n$, sức chứa W

Output: Giá trị tối ưu V^*

Sắp xếp vật phẩm giảm dần theo v_i/w_i $V^* \leftarrow 0$; $\text{rem} \leftarrow W$ **for** $i \leftarrow 1$ **to** n **do** **if** $\text{rem} \leq 0$ **then break**
if $w_i \leq \text{rem}$ **then** $x_i \leftarrow 1$; $V^* \leftarrow V^* + v_i$; $\text{rem} \leftarrow \text{rem} - w_i$ **else** $x_i \leftarrow \text{rem}/w_i$; $V^* \leftarrow V^* + v_i \cdot x_i$; $\text{rem} \leftarrow 0$
return V^*

- **Độ phức tạp:** $O(n \log n)$ – sắp xếp.
- **Tính chính xác:** Tối ưu cho Fractional, không tối ưu cho 0/1.

3.4 Quy hoạch động (Dynamic Programming)

Ý tưởng. Quy hoạch động [?] chia bài toán thành các bài toán con gộp nhau. Bảng $dp[i][w]$ = giá trị lớn nhất khi xét i vật phẩm đầu với sức chứa w .

Công thức truy hồi cho 0/1:

$$dp[i][w] = \begin{cases} 0 & \text{nếu } i = 0 \text{ hoặc } w = 0 \\ dp[i-1][w] & \text{nếu } w_i > w \\ \max(dp[i-1][w], dp[i-1][w-w_i] + v_i) & \text{ngược lại} \end{cases} \quad (10)$$

Công thức truy hồi cho Unbounded:

$$dp[w] = \max_{i: w_i \leq w} (dp[w-w_i] + v_i) \quad (11)$$

- **Độ phức tạp thời gian:** $O(nW)$ – pseudo-polynomial.
- **Độ phức tạp bộ nhớ:** 0/1: $O(nW)$; Unbounded: $O(W)$.
- **Tính chính xác:** Luôn cho nghiệm tối ưu.

Chiến lược	Thời gian	Bộ nhớ	Tối ưu?	Ghi chú
Vét cạn	$O(2^n)$	$O(n)$	Có	Chỉ khả thi $n \leq 20$
Quay lui	$O(2^n)$	$O(n)$	Có	Cắt tỉa cải thiện thực tế
Nhánh cận	$O(2^n)$	$O(n)$	Có	LP bound rất hiệu quả
Tham lam	$O(n \log n)$	$O(n)$	Không	Nhanh, nghiệm gần đúng
Quy hoạch động	$O(nW)$	$O(nW)$	Có	Cần W nguyên, không quá lớn

Bảng 3: So sánh các chiến lược cơ bản cho 0/1 Knapsack

4 Quy hoạch tuyến tính trong bài toán fractional knapsack

4.1 Chuẩn hóa bài toán và dạng chuẩn

Xét bài toán knapsack phân số dưới dạng quy hoạch tuyến tính:

$$\begin{aligned} & \text{maximize} && \sum_{i=1}^n v_i x_i \\ & \text{subject to} && \sum_{i=1}^n w_i x_i \leq W, \\ & && x_i \geq 0, \quad i = 1, \dots, n. \end{aligned}$$

Để đưa bài toán về dạng chuẩn dùng cho thuật toán đơn hình, ta đổi bài toán cực đại thành cực tiểu bằng cách đổi dấu hàm mục tiêu và thêm biến phụ x_{n+1} :

$$\begin{aligned} & \text{minimize} && f(x) = \sum_{i=1}^n (-v_i) x_i \\ & \text{s.t.} && \sum_{i=1}^n w_i x_i + x_{n+1} = W, \\ & && x_i \geq 0, \quad i = 1, \dots, n+1. \end{aligned}$$

Do x_{n+1} có hệ số bằng 1 trong ràng buộc duy nhất nên cột tương ứng tạo thành một cơ sở đơn vị. Vì vậy ta xác định ngay được một phương án cực biên khả thi ban đầu (PACB):

$$x_1 = x_2 = \dots = x_n = 0, \quad x_{n+1} = W.$$

4.2 Thuật toán đơn hình hai pha

Thuật toán đơn hình hai pha gồm:

- **Pha 1:** xây dựng bài toán phụ nhằm tìm PACB cho bài toán gốc.
- **Pha 2:** xuất phát từ PACB tìm được ở pha 1 để tối ưu hàm mục tiêu ban đầu.

Đối với fractional knapsack, biến phụ x_{n+1} đã tạo sẵn cơ sở đơn vị nên có thể bỏ qua Pha 1. Tuy nhiên, cấu trúc tổng quát vẫn được trình bày để mở rộng cho nhiều ràng buộc.

Các bước chi tiết Pha 1. Nếu chưa có PACB ban đầu, bổ sung các biến giả $x_{n+2}, \dots, x_{n+p}$ để tạo cơ sở đơn vị, rồi giải:

$$\text{minimize} \quad f(x) = \sum_{i=2}^p x_{n+i}.$$

Mục tiêu là đưa tất cả biến giả về 0.

Khởi tạo bảng đơn hình. Giả sử cơ sở hiện tại gồm các cột $\{A^1, A^2, \dots, A^m\}$:

hệ số	cơ sở	b	c_1	c_2	$\dots$	c_m	c_{m+1}	$\dots$	c_n
			x_1	x_2	$\dots$	x_m	x_{m+1}	$\dots$	x_n
c_1	A^1	$x_{1,0}$	1	0	$\dots$	0	$x_{1,m+1}$	$\dots$	$x_{1,n}$
c_2	A^2	$x_{2,0}$	0	1	$\dots$	0	$x_{2,m+1}$	$\dots$	$x_{2,n}$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
c_m	A^m	$x_{m,0}$	0	0	$\dots$	1	$x_{m,m+1}$	$\dots$	$x_{m,n}$
Δ			0	0	$\dots$	0	Δ_{m+1}	$\dots$	Δ_n

trong đó $\Delta_k = \sum_{j \in A} c_j x_{j,k} - c_k$ là **chi phí giảm** (reduced cost).

Chọn biến vào và biến ra cơ sở. Với bài toán cực tiểu, nếu tồn tại $\Delta_i > 0$ thì đưa x_i vào cơ sở có thể làm giảm hàm mục tiêu. Chọn biến vào với Δ_i lớn nhất, biến ra theo quy tắc tỉ lệ tối thiểu:

$$\min \left\{ \frac{x_{i,0}}{x_{i,k}} \mid x_{i,k} > 0 \right\}.$$

Pivot và cập nhật bảng. Lặp đến khi không còn $\Delta_i > 0$ (đạt tối ưu) hoặc không tồn tại biến ra (bài toán không bị chặn).

Pha 2. Khôi phục hàm mục tiêu gốc $f(x) = \sum_{i=1}^n (-v_i)x_i$ và tiếp tục đến khi đạt tối ưu.

Độ phức tạp. Trong trường hợp xấu nhất, số bước pivot có thể tăng theo hàm mũ (ví dụ Klee–Minty cube yêu cầu $\Theta(2^n)$ bước). Tuy nhiên trong thực tế thường rất nhanh.

4.3 Thuật toán đơn hình đối ngẫu

Thuật toán đơn hình đối ngẫu duy trì tính khả thi đối ngẫu và dần đưa nghiệm về khả thi nguyên thủy. Bắt đầu từ cơ sở thỏa $\Delta_i \leq 0$ (khả thi đối ngẫu) nhưng có thể $x_{i,0} < 0$ (chưa khả thi nguyên thủy), thuật toán pivot để loại các giá trị âm.

Các bước.

1. Khởi tạo cơ sở đối ngẫu khả thi với $\Delta_i \leq 0$.
2. Kiểm tra: nếu $x_{i,0} \geq 0, \forall i$ thì đạt tối ưu.
3. Chọn biến ra: hàng r với $x_{r,0} = \min_i x_{i,0}$.
4. Chọn biến vào: trên hàng r , xét các cột $a_{rj} < 0$, tính $\theta_j = \Delta_j / a_{rj}$, chọn j có θ_j nhỏ nhất.
5. Pivot và lặp đến khi tất cả phần tử cột về phải ≥ 0 .

4.4 Cập nhật nghiệm tối ưu khi bài toán thay đổi

Khi dữ liệu thay đổi nhỏ (dung lượng W , giá trị v_i , trọng lượng w_i , hoặc thêm ràng buộc), **re-optimization** tận dụng nghiệm cũ thay vì giải lại. Ba kịch bản chính:

1. Thay đổi dung lượng ($W \rightarrow W'$). Cơ sở cũ vẫn còn phù hợp. Cập nhật $x'_B = B^{-1}b'$. Nếu $x'_B \geq 0$, không cần pivot. Nếu có $x'_{B_i} < 0$, dùng Dual Simplex.

2. Thay đổi hệ số hàm mục tiêu ($v_i \rightarrow v'_i$). Tính lại Δ_j . Nếu vẫn thỏa điều kiện tối ưu, cơ sở vẫn tối ưu. Ngược lại, Primal Simplex tiếp tục pivot.

3. Thêm ràng buộc mới ($a^T x \leq \beta$). Nếu $a^T x^* \leq \beta$, nghiệm cũ vẫn khả thi. Ngược lại, dùng Dual Simplex: thêm hàng mới, đưa biến phụ vào cơ sở, pivot đối ngẫu cho đến khi khôi phục tối ưu.

4.5 Klee–Minty và tính mũ của đơn hình

Klee và Minty xây dựng họ bài toán **Klee–Minty cube** chứng minh đơn hình với quy tắc Dantzig có thể phải duyệt 2^n đỉnh:

$$\begin{aligned} \max \quad & c_1 x_1 + c_2 x_2 + \cdots + c_n x_n \\ \text{s.t.} \quad & x_1 \leq u_1, \\ & \varepsilon x_1 + x_2 \leq u_2, \\ & \varepsilon x_1 + \varepsilon x_2 + x_3 \leq u_3, \\ & \vdots \\ & \varepsilon x_1 + \cdots + \varepsilon x_{n-1} + x_n \leq u_n, \\ & x_i \geq 0, \end{aligned}$$

với $0 < \varepsilon < 1$ đủ nhỏ. Hệ quả: đơn hình không phải thuật toán đa thức trong trường hợp xấu nhất, nhưng vẫn rất nhanh trên dữ liệu thực tế.

5 Quy hoạch nguyên trong bài toán unbounded knapsack và 0/1 knapsack

5.1 Ý tưởng và mô hình toán học

Bài toán 0/1 Knapsack dưới dạng Integer Linear Programming (ILP):

$$\begin{aligned} & \text{maximize} && \sum_{i=1}^n v_i x_i \\ & \text{s.t.} && \sum_{i=1}^n w_i x_i \leq W, \\ & && x_i \in \{0, 1\}. \end{aligned}$$

Cách tiếp cận phổ biến: giải **LP relaxation** ($0 \leq x_i \leq 1$) để có cận trên, rồi áp dụng nhánh-cận hoặc cutting planes để tìm nghiệm nguyên.

5.2 Phương pháp nhánh-cận (Branch and Bound)

Phân chia không gian nghiệm bằng cách thêm ràng buộc trên biến phân số. Tại mỗi nút: giải LP relaxation cho cận trên, dùng nghiệm nguyên tốt nhất làm cận dưới, loại nút không cải thiện được.

Algorithm 5.1 Branch-and-Bound cho knapsack 0/1

Input: Bài toán knapsack 0/1 với n đồ vật và dung lượng W

Output: Nghiệm nguyên tối ưu

Giải LP relaxation ban đầu $\rightarrow$ nghiệm x^* , giá trị UB **if** x^* nguyên **then return** x^*

Khởi tạo tập node chưa xử lý; đưa node gốc vào Khởi tạo $best_value$ từ một nghiệm nguyên khả thi **while** còn node chưa xử lý **do** Chọn node tiếp theo (DFS/BFS/best-first) Giải LP relaxation của node $\rightarrow UB_{node}$ **if** $UB_{node} \leq best_value$ **then** Loại node (prune) **else if** nghiệm LP hiện tại nguyên **then** Cập nhật $best_value$ **else** Chọn biến phân số x_k , tạo 2 node con $x_k = 0$ và $x_k = 1$ **return** nghiệm nguyên tốt nhất

Độ phức tạp tồi nhất $O(2^n)$ nhưng thường rất nhanh nhờ cận trên LP chặt.

5.3 Lát cắt Gomory

Sinh ràng buộc tuyến tính hợp lệ loại nghiệm phân số hiện tại nhưng giữ mọi nghiệm nguyên. Từ hàng i của bảng đơn hình với b_i^* phân số, sinh Gomory cut:

$$\sum_{j \in N} f(\bar{a}_{ij}) x_j \geq f(b_i^*),$$

trong đó $f(\alpha) = \alpha - \lfloor \alpha \rfloor$.

Các bước:

1. Giải LP relaxation.
2. Nếu nghiệm nguyên, dừng.
3. Chọn hàng có vẻ phải phân số.
4. Sinh Gomory cut từ hàng đó, thêm vào hệ.
5. Giải lại LP. Lặp đến khi nghiệm nguyên.

Phương pháp này thường được kết hợp với branch-and-bound thành **branch-and-cut**, là nền tảng các bộ giải ILP hiện đại.

6 Thực nghiệm, so sánh và đánh giá

Phần này trình bày quy trình thực nghiệm toàn diện, từ việc sinh dữ liệu, thiết lập khung benchmark cách ly tiến trình, cho đến phân tích chi tiết 10 thuật toán. Toàn bộ mã nguồn được triển khai bằng Python 3.14.

6.1 Phương pháp sinh dữ liệu và thiết lập thực nghiệm

Cấu hình hệ thống: bộ vi xử lý Intel Core i7-11800H (2.30 GHz, 8 nhân 16 luồng), RAM 16GB, Windows 11. Python 3.14 với NumPy 2.x và SciPy 1.x.

Quy trình thực nghiệm (Pipeline)

Pipeline gồm 4 pha tuần tự:

1. **Pha 1 – Sinh dữ liệu:** Đọc `test_scenarios.json`, sinh các instance Knapsack có kiểm soát thuộc tính thống kê.
2. **Pha 2 – Kiểm tra chất lượng:** Xác minh tham số thực tế của dữ liệu sinh ra so với mục tiêu.
3. **Pha 3 – Khung Benchmark:** Chạy các thuật toán trên tiến trình con cô lập, đo wall-clock time và peak memory, xử lý timeout 5 giây và OOM.
4. **Pha 4 – Phân tích & Trực quan:** Trực quan hóa bằng scatter plots và curve fitting.

Thuật toán sinh dữ liệu (Data Generation)

Để tránh overfitting vào lưới tham số cố định, dùng 2 kỹ thuật chính: **Gaussian jitter** và **phân rã Cholesky**.

Gaussian Jitter.

$$n_{\text{actual}} = \max\left(2, \text{round}(\mathcal{N}(n_{\text{anchor}}, \sigma_n))\right), \quad \sigma_n = \max(1.0, 0.35n_{\text{anchor}}) \quad (12)$$

$$r_{\text{actual}} = \text{clip}(\mathcal{N}(r_{\text{anchor}}, \sigma_r), 0.01, 1.0), \quad \sigma_r = \max(0.005, 0.15r_{\text{anchor}}) \quad (13)$$

Tương quan bằng Cholesky. Sinh $x, z \sim \mathcal{N}(0, 1)$ độc lập, tạo:

$$y = \rho \cdot x + \sqrt{\max(0, 1 - \rho^2)} \cdot z \quad (14)$$

Để $w_i \in [1, W_{\max}]$ nguyên, chuẩn hóa min-max:

$$x_i^{\text{norm}} = \frac{x_i - \min(x)}{\max(x) - \min(x)}, \quad w_i = \text{round}(x_i^{\text{norm}} \cdot (W_{\max} - 1) + 1) \quad (15)$$

Khi $\rho \geq 0.9$: dùng Rejection Sampling tối đa 200 lần để đạt $|r_{\text{actual}} - \rho| \leq 0.03$.

Kịch bản kiểm thử. 4 kịch bản với tổng **822 instances**:

Bảng 4: Cấu hình 4 kịch bản kiểm thử.

Kịch bản	n_{anchor}	$W / \sum w$	ρ	Số mẫu	Tổng instance
core_exact	15, 30, 45	0.1, 0.5, 0.9	0, 0.5, 0.95	15	405
core_heuristic	250, 500, 750	0.1, 0.5, 0.9	0, 0.5, 0.95	15	405
stress	50	0.5	0, 0.95	3	6
extreme_n	1000, 5000, 10000	0.5	0	2	6
Tổng cộng					822

Xác minh chất lượng sinh dữ liệu

Phân tích biểu đồ QA xác nhận: phân phối n_{actual} tuân theo Gaussian quanh anchor; tỉ lệ capacity thực tế tách thành 3 cụm rõ ràng; tương quan Pearson thực tế bám sát mục tiêu với $n \geq 100$ và dao động theo phương sai mẫu nhỏ khi $n = 15$ (sai số chuẩn $1/\sqrt{n-2} \approx 0.28$, khoảng tin cậy 95% $[-0.55, 0.55]$).

Khung Benchmark

Kiến trúc cách ly tiến trình bằng multiprocessing.Process:

- **Cô lập OOM:** Lỗi bộ nhớ ở tiến trình con không sập tiến trình chính.
- **Timeout cưỡng bức:** process.join(timeout) cho dừng chính xác 5 giây.
- **Đo bộ nhớ chính xác:** tracemalloc đo heap riêng biệt.

Tổng cộng $822 \text{ instances} \times 10 \text{ thuật toán} = \mathbf{8.220 \text{ lượt chạy}}$.

Bảng 5: Danh sách 10 thuật toán được benchmark.

#	Thuật toán	Biến thể	Độ phức tạp lý thuyết
1	Quy hoạch động (DP)	0/1	$\mathcal{O}(n \cdot W)$
2	DP Unbounded	Unbounded	$\mathcal{O}(n \cdot W)$
3	Nhánh cận (BranchAndBound)	0/1	$\mathcal{O}(2^n)$
4	Quay lui (Backtracking)	0/1	$\mathcal{O}(2^n)$
5	Tham lam 0/1 (Greedy01)	0/1 (Xấp xỉ)	$\mathcal{O}(n \log n)$
6	Tham lam Fractional	Fractional	$\mathcal{O}(n \log n)$
7	Primal Simplex	Fractional (LP)	Đa thức trung bình
8	Dual Simplex	Fractional (LP)	Đa thức trung bình
9	Gomory Cut	0/1 (ILP)	Hữu hạn
10	Simplex B&B	0/1 (ILP)	$\mathcal{O}(2^n)$

6.2 Kết quả thực nghiệm

Trong 8.220 lượt chạy: 4.790 SUCCESS (58.3%), 3.378 TIMEOUT (41.1%), 52 ERROR (0.6%).

Phân tích thời gian

Bảng 6: Thời gian thực thi (giây) – chỉ tính SUCCESS.

Thuật toán	Số SUCCESS	Trung vị	Trung bình	Tối đa	Timeout %
Greedy01	822	0.00016	0.00039	0.01300	0.0%
GreedyFractional	820	0.00020	0.00054	0.00997	0.2%
BranchAndBound	795	0.00165	0.09579	2.60702	0.1%
Backtracking	338	0.00188	0.20254	4.38516	55.7%
DPUnbounded	501	0.07983	0.50540	4.86327	39.1%
DP	464	0.12194	0.51069	4.33586	43.6%
DualSimplex	407	0.18939	0.58592	4.85935	50.5%
PrimalSimplex	408	0.18982	0.59840	4.89048	50.4%
GomoryCut	113	0.11846	0.65365	4.78231	86.3%
SimplexBnB	122	0.61336	1.28224	4.88688	85.2%

Phân tích chuyên sâu thuật toán chính xác

BranchAndBound vs Backtracking. Cùng $\mathcal{O}(2^n)$ tồi nhất nhưng Backtracking timeout 55.7%, BranchAndBound chỉ 0.1%. Lý do: chất lượng cận trên.

- Backtracking dùng suffix sum: $\text{bound} = \text{value}_{\text{current}} + \sum_{j=\text{level}}^n v_j$, bỏ qua W .
- BranchAndBound sắp xếp theo density, dùng LP relaxation bound: $\text{bound} = \text{value}_{\text{current}} + \sum_{j=\text{level}}^{k-1} v_j + v_k \cdot \frac{W_{\text{rem}}}{w_k}$, tích hợp cả sức chứa và tối ưu cục bộ tham lam.

Pseudo-polynomial của DP. Trong stress ($n = 50, W_{\text{max}} = 100.000, W \approx 50.000$), bảng DP có $(n+1) \times (W+1) \approx 2.5 \times 10^6$ ô, chạy vòng lặp Python thuần hơn 5 giây, dẫn đến 100% timeout. Khi W lớn hơn 2^n , DP chậm hơn cả BranchAndBound combinatorial (chỉ 1.21 ms trên stress).

Bottleneck Simplex/ILP tự triển khai. Gomory Cut và Simplex B&B timeout 86.3% và 85.2% do:

- **Gaussian Elimination cho bộ lọc ràng buộc dư thừa:** Với $n = 750, m = 751$, số phép tính $\approx 4.2 \times 10^8$ trên mỗi nút.
- **Sự hội tụ chậm của Gomory Cut:** Sai số floating-point làm Dual Simplex lặp hàng trăm lần không hội tụ về nghiệm nguyên.

So sánh bộ nhớ

Bảng 7: Bộ nhớ đỉnh (MB) – chỉ tính SUCCESS.

Thuật toán	Trung vị	Trung bình	Tối đa
Greedy01	0.0033	0.0141	0.5166
Backtracking	0.0040	0.0097	0.1186
GreedyFractional	0.0057	0.0328	0.6515
BranchAndBound	0.0070	0.0173	0.0974
SimplexBnB	0.0296	0.0406	0.3405
PrimalSimplex	0.0577	0.0797	0.3229
DualSimplex	0.0642	0.0883	0.3618
GomoryCut	0.0731	0.1567	1.0171
DPUnbounded	0.2646	0.4015	3.0922
DP	2.8169	11.1974	88.8089

Phát hiện chính: DP đạt 88.8 MB do bảng 2D $(n + 1) \times (W + 1)$. DPUnbounded chỉ 3.09 MB nhờ giữ 1 hàng kích thước $W + 1$ – tiết kiệm 28 lần.

Curve fitting độ phức tạp thực nghiệm

Khớp 4 mô hình (linear, quadratic, cubic, exponential) chọn theo R^2 cao nhất:

Bảng 8: Khớp đường cong cho các thuật toán tiêu biểu.

Thuật toán	Mô hình khớp tốt nhất	R^2	Phù hợp lý thuyết?
Greedy01	Cubic	0.98	Lý thuyết $\mathcal{O}(n \log n)$
GreedyFractional	Cubic	0.82	Lý thuyết $\mathcal{O}(n \log n)$
DP	Cubic	0.79	Khá khớp $\mathcal{O}(n \cdot W)$
BranchAndBound	Cubic	0.50	Hiệu ứng cắt nhánh tốt

Giải thích: Trên dải $n \leq 750$, $n \log n$ rất khó phân biệt với đa thức bậc ba. Constant factors nhỏ và overhead khởi động tiến trình khiến mô hình cubic xấp xỉ tốt hơn. BranchAndBound khớp cubic (không phải exponential) chứng tỏ LP bound cắt số trạng thái từ 2^n xuống mức đa thức trên hầu hết instances.

Phân tích độ nhạy: warm-start Dual Simplex

Phân tích độ nhạy nghiên cứu sự biến động nghiệm tối ưu khi tham số đầu vào thay đổi. **Warm-start** tận dụng bảng tối ưu hiện tại làm xuất phát điểm thay vì giải lại từ đầu, bảo toàn tính khả thi gốc/đối ngẫu của cơ sở cũ. Thực nghiệm trên 200 bài toán Knapsack ($N \leq 250$) với 3 kịch bản:

Kịch bản 1 – Capacity (RHS): $W \rightarrow W + 0.15W$. Khi b thay đổi: $x_B = B^{-1}(b + \Delta b)$. Nếu $x_B \geq 0$, không cần pivot. Ngược lại, $\bar{c}^T \geq 0$ vẫn được bảo toàn (không phụ thuộc b), dùng **Dual Simplex** xoay pivot trực tiếp.

Kịch bản 2 – Value (objective): $c_0 \rightarrow c_0 + 0.25c_0$. Nếu $j \notin B$: $\bar{c}'_j = \bar{c}_j + \Delta c_j$. Nếu $j \in B$: $x_B = B^{-1}b \geq 0$ vẫn khả thi nhưng có thể mất tính tối ưu đối ngẫu nếu $\bar{c}'_k < 0$. Dùng **Primal Simplex** pivot từ bảng hiện tại.

Kịch bản 3 – Volume constraint: Thêm ràng buộc $\sum v_i x_i \leq V_{\max}$. Đưa biến bù $x_{\text{slack}, m+1} = b_{m+1} - a_{m+1}^T x^*$ vào cơ sở. Nếu < 0 , mất khả thi gốc nhưng giữ tính đối ngẫu. Dùng **Dual Simplex** đưa biến bù âm ra khỏi cơ sở.

Kết quả thực nghiệm:

- **Capacity:** Trung bình **4.3 pivots** vs **60.1 pivots** (Re-solve). Speedup **24.2**× (16.2ms $\rightarrow$ 0.92ms).
- **Value:** Warm-start trung bình **0.37 pivots** (85% trường hợp = 0 pivots) vs **57.1 pivots**. Speedup kỷ lục **770**× (15.5ms $\rightarrow$ 0.04ms).
- **Volume:** Trung bình **21.2 pivots** vs **48.1 pivots**. Speedup **5.6**× (12.1ms $\rightarrow$ 4.26ms).

Thực nghiệm chứng minh rằng cấu trúc cơ sở (basis) đóng vai trò bộ nhớ lưu trữ thông tin mạnh mẽ, cho phép giải quyết hiệu quả bài toán biến động – nền tảng cho branch-and-bound và cutting plane.

6.3 Thảo luận

1. **Trade-off chính xác vs tốc độ:** Greedy01 nhanh nhất (0.16ms trung vị, 0% timeout, kể cả $n = 10.000$) nhưng không tối ưu. BranchAndBound chính xác tốt nhất (1.65ms trung vị) nhờ LP bound chặt.
2. **Giới hạn của LP/ILP tổng quát:** Knapsack chỉ có 1 ràng buộc capacity, BranchAndBound combinatorial khai thác cấu trúc đặc biệt ($\mathcal{O}(n)$ tính cận). Simplex phải duy trì toàn bộ bảng ma trận và lặp filter Gaussian. Chi phí mỗi nút Simplex lớn hơn hàng nghìn lần.
3. **Ưu thế Simplex khi W lớn:** Trong `core_heuristic`, Simplex chỉ 1ms trung vị (khi SUCCESS), nhanh hơn DP $1800\times$ (1.86s). Simplex độc lập với W – hiệu quả khi W rất lớn.

LP/ILP có ưu thế khi: bài toán Knapsack đa chiều (DP bùng nổ); Fractional Knapsack (Simplex cho nghiệm chính xác + thông tin đối ngẫu); cần sensitivity analysis; W rất lớn và n vừa phải.

7 Kết luận

Nghiên cứu này đã đánh giá định lượng 10 thuật toán giải bài toán Knapsack trên 822 instances (8.220 lượt chạy).

Phát hiện chính:

- **BranchAndBound combinatorial** tốt nhất cho 0/1: 1.65ms trung vị, 99.9% thành công.
- **Quy hoạch động** bị giới hạn bởi W , timeout 100% khi $W = 50.000$. DPUnbounded tiết kiệm bộ nhớ $28\times$ ($88.8\text{MB} \rightarrow 3.09\text{MB}$).
- **LP/ILP tự triển khai** gặp bottleneck Gaussian elimination Python thuần, timeout 50.4–86.3%.
- **Các thuộc tính đầu vào độc lập thống kê** (correlation chéo ≈ 0) nhưng có tác động phi tuyến mạnh lên hiệu năng: N chi phối hàm mũ/đa thức, capacity ratio là bottleneck của pseudo-polynomial, Pearson r quyết định hiệu quả cắt nhánh.

Hạn chế:

1. Môi trường Python thuần, chưa phản ánh tốc độ tối đa khi biên dịch C/C++.
2. Chưa so sánh trực tiếp với Gurobi, COIN-OR CBC, SciPy linprog.
3. Chưa đối chiếu với bộ test chuẩn Pisinger.

Hướng phát triển:

1. Triển khai Simplex bằng Cython/Numba và hot-start với Dual Simplex.
2. Mở rộng benchmark sang bài toán đa chiều có ràng buộc phức tạp, nơi LP thực sự thể hiện ưu thế.